

Compte rendu de la semaine 4

Sujet du stage : Classification non supervisée d'une image satellite hyperspectrale

Travail effectué durant la semaine

Expérience 1

L'objectif de cette étape est d'améliorer les performances de la classification non supervisée (K-means) en intégrant une information spatiale locale dans la représentation des pixels

```
# Dans l'approche classique, chaque pixel est représenté uniquement par sa
# signature spectrale (200 bandes). Cette représentation ne tient pas compte
# des relations spatiales entre pixels voisins.
#
# Afin d'introduire une information spatiale locale, chaque pixel est décrit
# par sa propre signature spectrale ainsi que par celles de ses quatre voisins
# directs (haut, bas, gauche et droite).
#
# Un padding par réflexion (reflect padding) est appliqué sur les bords de
# l'image afin de pouvoir extraire les voisins pour tous les pixels tout en
# conservant la taille originale de l'image.
#
# Pour chaque pixel, un nouveau vecteur de caractéristiques est construit par
# concaténation des signatures spectrales du pixel central et de ses voisins :
#
# [centre, haut, bas, gauche, droite]
#
# Chaque signature contenant 200 bandes spectrales, le vecteur final possède
# 1000 caractéristiques (5 x 200).
#
# L'objectif est d'enrichir la représentation des pixels en ajoutant un contexte
# spatial local afin d'obtenir des régions de classification plus cohérentes et
# de réduire l'apparition de pixels isolés dans les résultats du K-means.
```

```

# Preprocessing
M, N, K = data.shape

data_pad = np.pad(data, ((1,1),(1,1),(0,0)), mode='reflect')
features = []

for i in range(1, M+1):
    for j in range(1, N+1):
        centre = data_pad[[i,j,:]]
        haut = data_pad[i-1,j,:]
        bas = data_pad[i+1,j,:]
        gauche = data_pad[i,j-1,:]
        droite = data_pad[i,j+1,:]

        feature = np.concatenate([centre, haut, bas, gauche, droite])
        features.append(feature)

X = np.array(features)

```

Chaque pixel est ainsi décrit par un vecteur de 1000 caractéristiques obtenu par concaténation de cinq signatures spectrales de 200 bandes. Cette approche vise à exploiter la corrélation spatiale entre pixels voisins afin d'améliorer la cohérence géométrique des régions obtenues par K-means et de limiter la présence de pixels isolés.

```

# K-means clustering
n_clusters = 7 # à tester
kmeans = KMeans(n_clusters=n_clusters, random_state=0, n_init=20)
labels = kmeans.fit_predict(X)

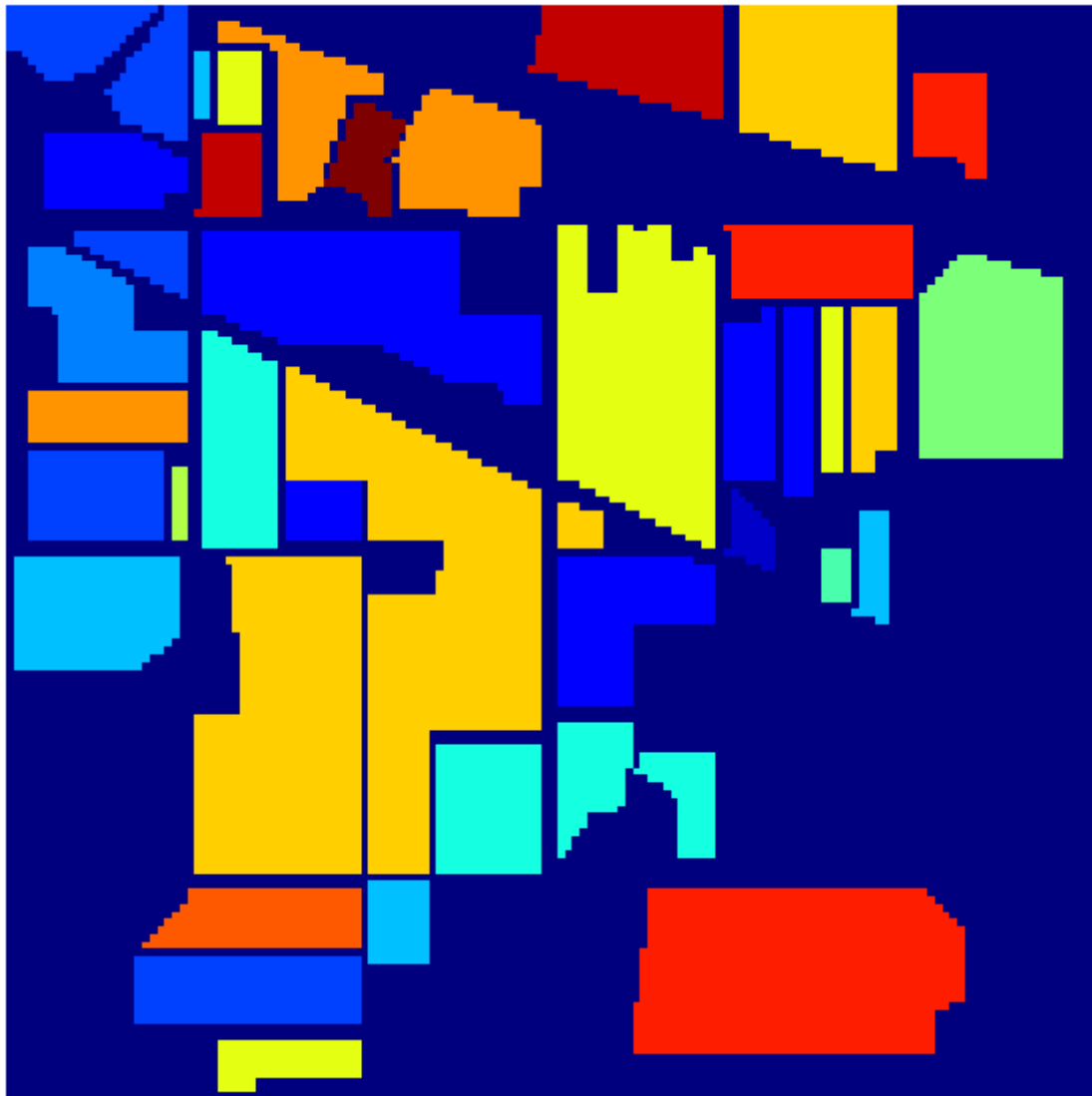
# Reconstruction image
classification = labels.reshape(M, N)

```

L'ensemble des pixels est ensuite transformé en une matrice (145×145, 1000)

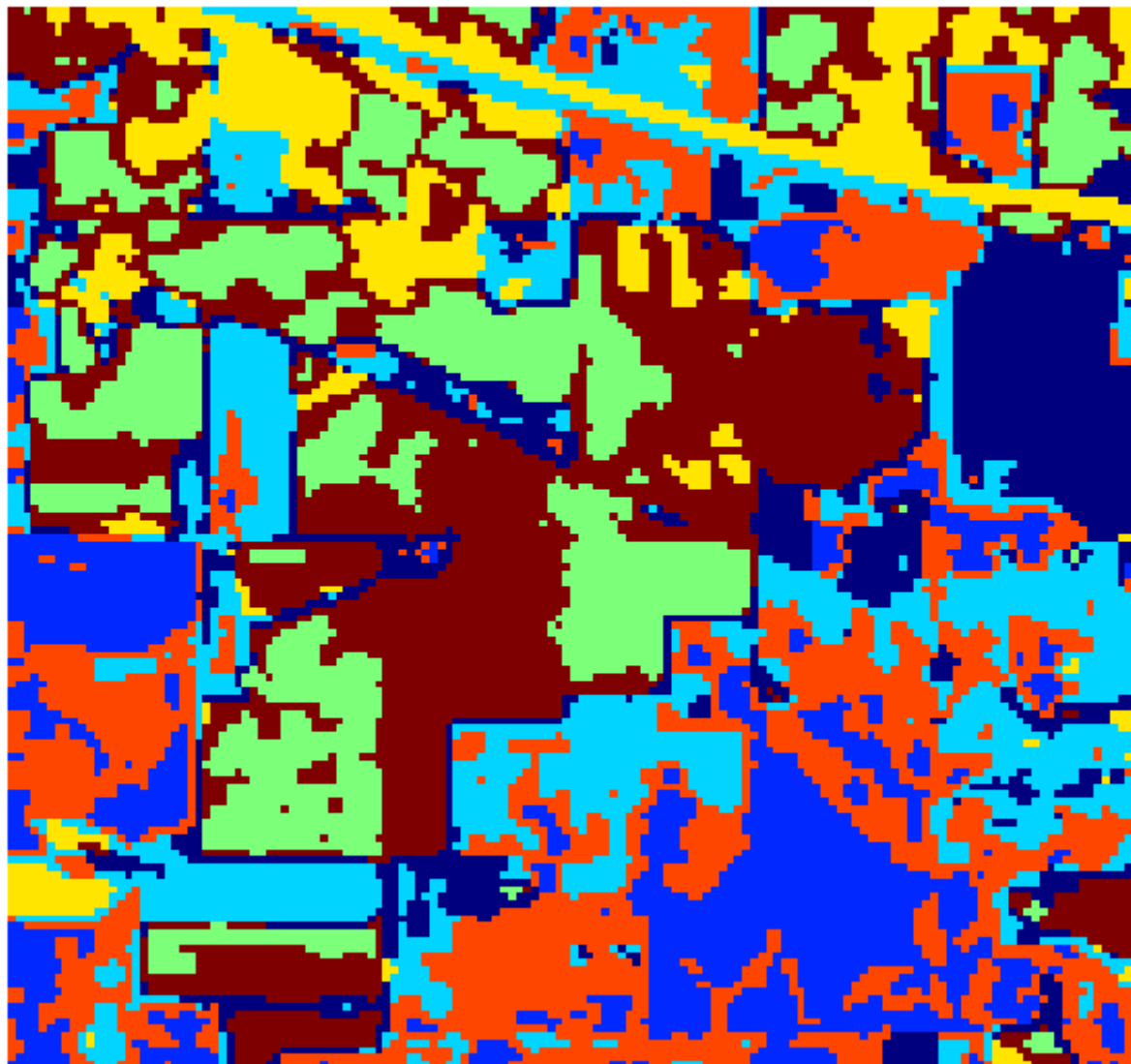
Cette matrice est utilisée comme entrée de K-means pour effectuer la classification.

Ground Truth



Simulation avec 7 clusters (K = 7 meilleur compromis actuel)

K-means Classification (7 clusters)

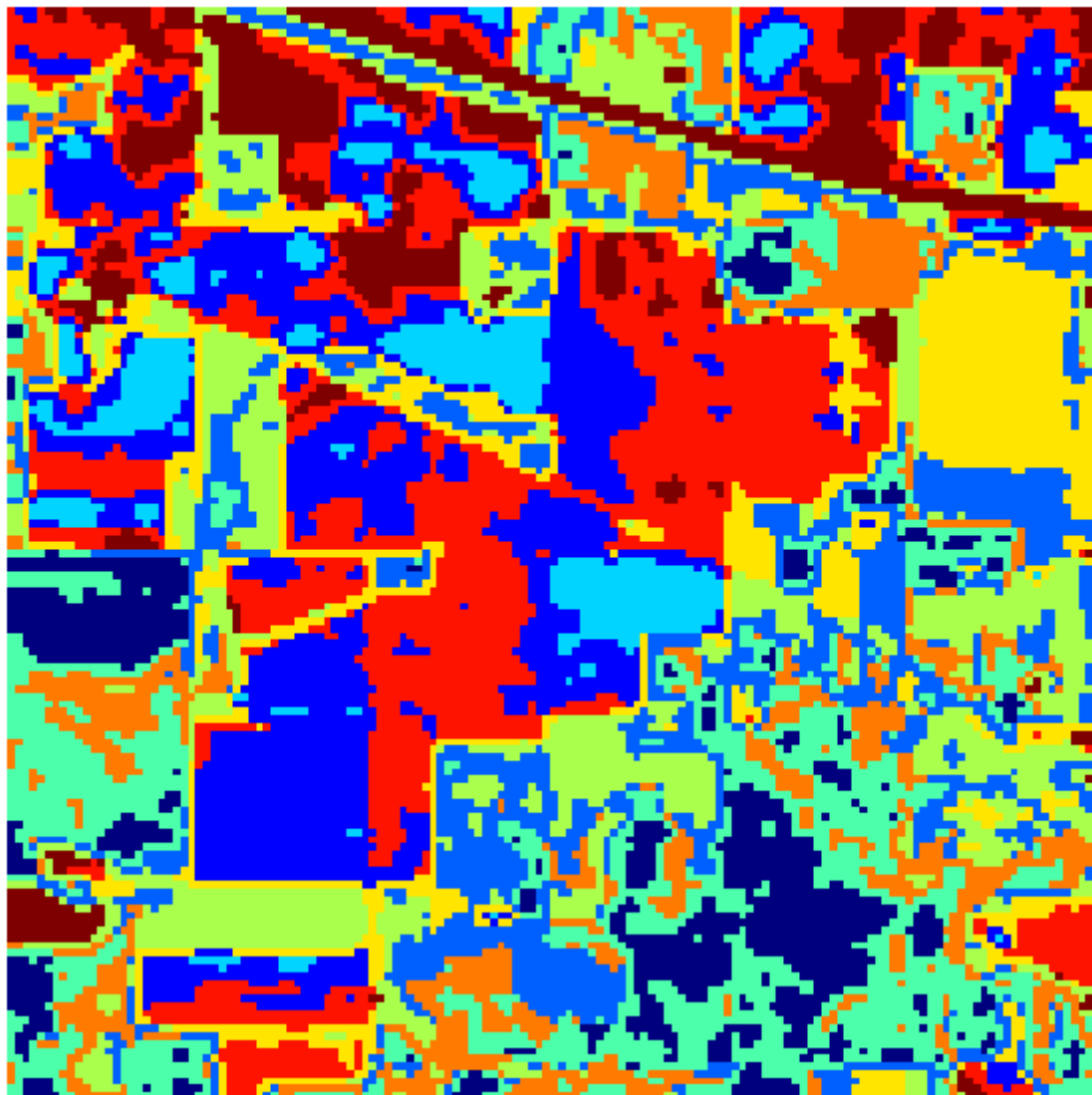


Accuracy obtenue : 0,4250

Cette configuration fournit la meilleure accuracy parmi les simulations réalisées. Les régions obtenues sont un moins homogènes mais présentent moins de bruit spatial que la version classique du K-means basée uniquement sur les signatures spectrales. Cependant, plusieurs classes de la vérité terrain sont regroupées dans un même cluster.

Simulation avec 10 clusters

K-means Classification (10 clusters)

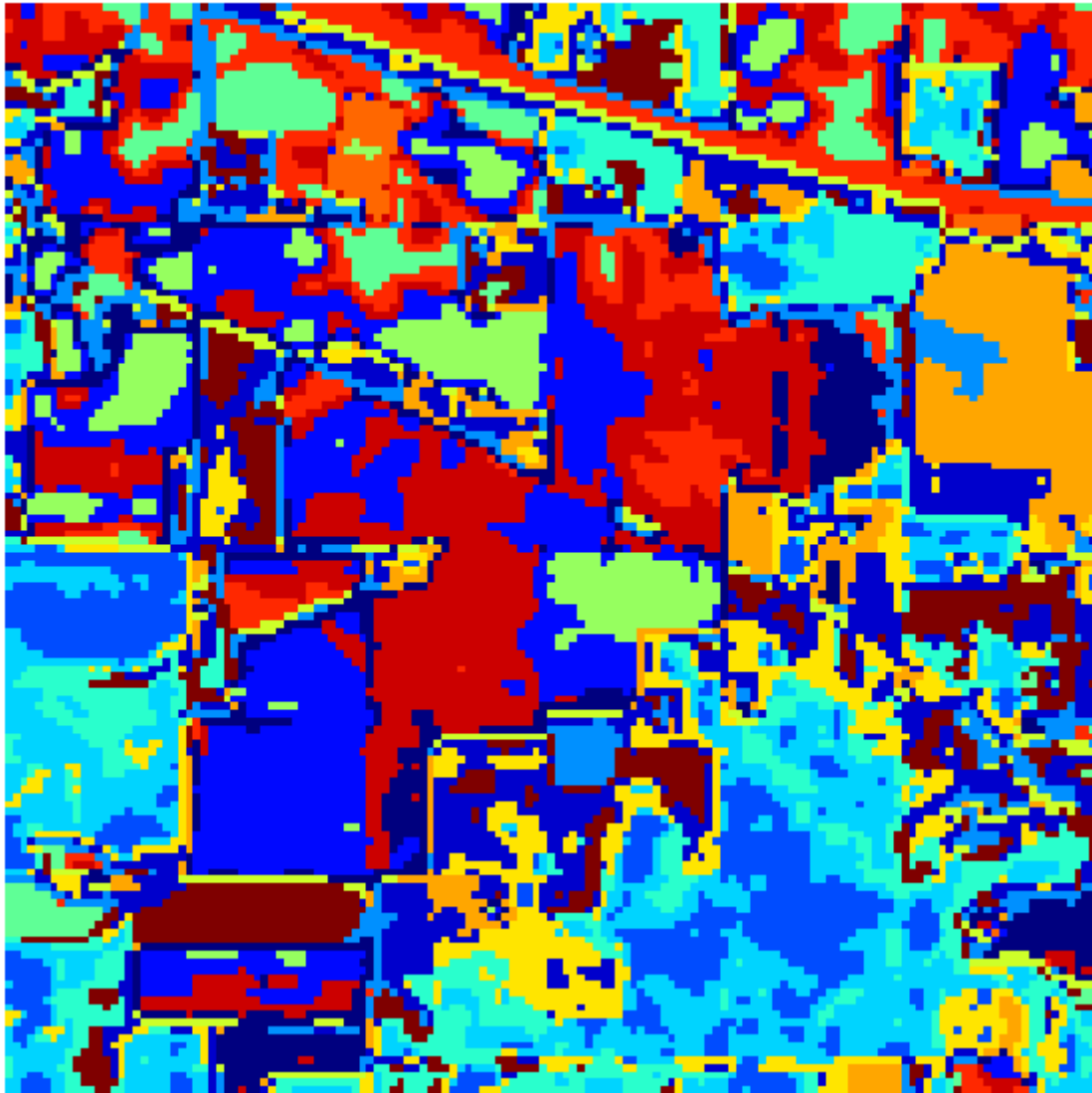


Accuracy obtenue : 0,3962

L'augmentation du nombre de clusters permet de faire apparaître davantage de structures présentes dans la vérité terrain. Certaines régions commencent à être mieux distinguées et les frontières entre zones deviennent plus visibles. En revanche, la segmentation devient également plus fragmentée, ce qui explique la légère diminution de l'accuracy par rapport à $K=7$.

Simulation avec 16 clusters (aligné avec le nombre de classe du gt)

K-means Classification (16 clusters)



Accuracy obtenue : 0,3760

Cette valeur correspond au nombre de classes présentes dans la vérité terrain. Bien que l'accuracy soit plus faible que pour les valeurs précédentes, l'analyse visuelle montre que plusieurs régions obtenues se rapprochent davantage de la structure observée dans la vérité terrain. Certaines frontières et certaines zones homogènes apparaissent plus clairement, ce qui suggère que le nombre plus élevé de clusters permet de mieux représenter la diversité des classes réelles. Ainsi, l'accuracy seule ne reflète pas complètement la qualité visuelle de la segmentation obtenue.

Conclusion de l'expérience

Les résultats montrent que l'introduction du voisinage spatial permet d'améliorer la cohérence géométrique des cartes de classification obtenues par K-means. L'accuracy maximale est obtenue pour $K=7$, ce qui indique que cette valeur fournit la meilleure correspondance globale avec la vérité terrain selon la métrique utilisée.

Cependant, l'analyse visuelle révèle que les classifications obtenues avec un nombre plus élevé de clusters, notamment $K=16$, reproduisent mieux certaines structures spatiales observées dans la vérité terrain, malgré une accuracy faible.

Expérience 2 :

Cette deuxième simulation a été réalisée dans le prolongement des documents que vous m'avez fournis. Je me suis directement inspiré des recommandations méthodologiques proposées, notamment concernant la normalisation des bandes spectrales et la préparation des données, la mesure de vraisemblance afin de reproduire et tester les approches de classification non supervisée sur l'image hyperspectrale.

Centrer et normaliser chaque bande spectrale

```
def normalize(data):  
    """Normalise chaque bande spectrale pour avoir moyenne 0, écart-type 1."""  
    data_norm = np.zeros_like(data, dtype=np.float64)  
    for k in range(data.shape[2]):  
        band = data[:, :, k].astype(np.float64)  
        mu = band.mean()  
        sig = band.std()  
        if sig > 0:  
            data_norm[:, :, k] = (band - mu) / sig  
        else:  
            data_norm[:, :, k] = 0.0  
    return data_norm  
  
data_norm = normalize(data)
```

Cette fonction de normalisation est inspirée de la recommandation présente dans le document **discussion_N260603.pdf**, où il est mentionné que les bandes spectrales doivent être centrées et réduites afin de stabiliser les algorithmes de classification. Chaque bande est ainsi transformée pour avoir une moyenne nulle et un écart-type égal à 1, ce qui permet d'assurer une contribution équilibrée de toutes les longueurs d'onde dans le processus de clustering.

$$\mu_k = \frac{1}{MN} \sum_{m,n} I_{m,n,k} \text{ et } \sigma_k = \sqrt{\frac{1}{MN} \sum_{m,n} (I_{m,n,k} - \mu_k)^2}$$

$$I'_{m,n,k} = \frac{I_{m,n,k} - \mu_k}{\sigma_k}$$

Extraction de caractéristiques spatiales et pondération spectrale

```
def build_features(data, weights=None):
    """
    Pour chaque pixel : concatène [centre, haut, bas, gauche, droite].
    weights : vecteur de longueur K pour pondérer les bandes (paramètres c_k).
    |         | Si None, toutes les bandes ont le même poids.
    """
    M, N, K = data.shape

    if weights is not None:
        # Applique c_k à chaque bande
        data = data * weights[np.newaxis, np.newaxis, :]

    data_pad = np.pad(data, ((1,1),(1,1),(0,0)), mode="reflect")
    features = []

    for i in range(1, M+1):
        for j in range(1, N+1):
            centre = data_pad[i, j, :]
            haut   = data_pad[i-1, j, :]
            bas    = data_pad[i+1, j, :]
            gauche = data_pad[i, j-1, :]
            droite = data_pad[i, j+1, :]
            features.append(np.concatenate([centre, haut, bas, gauche, droite]))

    return np.array(features)
```


Pour chaque pixel, on construit un descripteur combinant l'information spectrale du pixel central ainsi que celle de ses voisins immédiats (haut, bas, gauche, droite), ce qui permet d'introduire une cohérence spatiale dans la représentation.

Une pondération optionnelle des bandes spectrales peut être appliquée via un vecteur de paramètres ckc_kck , permettant de moduler l'importance de chaque longueur d'onde dans le processus de classification.

Classification par K-means

```
def run_kmeans(X, n_clusters, seed=42):  
  
    rng = np.random.RandomState(seed)  
    init_idx = rng.choice(len(X), n_clusters, replace=False)  
    init_centers = X[init_idx]  
  
    kmeans = KMeans(n_clusters=n_clusters, init=init_centers, n_init=1, random_state=seed  
    )  
    labels = kmeans.fit_predict(X)  
    return labels.reshape(M, N)
```

Les centres initiaux sont sélectionnés à l'aide d'une graine aléatoire fixe, puis utilisés comme point de départ unique de l'algorithme. Cette approche permet d'obtenir des résultats identiques d'une exécution à l'autre pour un même ensemble de paramètres. Elle est utilisée dans le cadre de l'optimisation des coefficients spectraux ck , car elle garantit que les variations observées dans les performances proviennent des paramètres étudiés et non d'une initialisation aléatoire différente du K-means.

Calcul de l'indicateur CE

```
def compute_CE(classification):  
    """  
    CE : proportion de pixels dont au moins un voisin (4-connexe)  
    appartient à une classe différente.  
    Valeur entre 0 et 1. On cherche à MINIMISER CE.  
    """  
    # Décalages dans les 4 directions  
    up    = np.roll(classification, 1, axis=0)  
    down  = np.roll(classification, -1, axis=0)  
    left  = np.roll(classification, 1, axis=1)  
    right = np.roll(classification, -1, axis=1)  
  
    # Un pixel est sur un contour si au moins un voisin diffère  
    edge = (  
        (classification != up) |  
        (classification != down) |  
        (classification != left) |  
        (classification != right)  
    )  
    return edge.mean()
```

Afin d'évaluer la cohérence spatiale de la classification obtenue, un indicateur CE (Contour Estimator) a été implémenté. Pour chaque pixel, la classe attribuée est comparée à celles de ses quatre voisins directs (haut, bas, gauche, droite). Un pixel est considéré comme appartenant à un contour lorsqu'au moins un de ses voisins appartient à une classe différente. L'indicateur CE correspond alors à la proportion de pixels situés sur un contour dans l'image. Une faible valeur de CE traduit une classification composée de régions plus homogènes et spatialement cohérentes.

Calcul de l'indicateur CIP

```
def compute_CIP(classification):  
    """  
    CIP : proportion de pixels isolés (tous les 8 voisins dans une autre classe).  
    Valeur entre 0 et 1. On cherche à MINIMISER CIP.  
    """  
  
    pad = np.pad(classification, 1, mode='edge')  
    isolated = np.ones((M, N), dtype=bool)  
  
    for di in [-1, 0, 1]:  
        for dj in [-1, 0, 1]:  
            if di == 0 and dj == 0:  
                continue  
            neighbor = pad[1+di:M+1+di, 1+dj:N+1+dj]  
            isolated &= (classification != neighbor)  
  
    return isolated.mean()
```

L'indicateur CIP (Count of Isolated Pixels) mesure la proportion de pixels isolés dans la classification. Un pixel est considéré comme isolé lorsque l'ensemble de ses huit voisins appartient à une classe différente. Cette mesure permet d'évaluer le niveau de bruit spatial présent dans la classification obtenue. Une faible valeur de CIP traduit une meilleure cohérence spatiale des régions détectées, tandis qu'une valeur élevée indique la présence de nombreux pixels aberrants ou isolés.

Construction d'une mesure de vraisemblance

```
def compute_vraisemblance(classification):  
    """  
    Mesure combinée : produit de (1 - CE) et (1 - CIP).  
    Interprétée comme une probabilité. On cherche à MAXIMISER.  
    """  
  
    ce = compute_CE(classification)  
    cip = compute_CIP(classification)  
    return (1 - ce) * (1 - cip)
```

Conformément aux pistes proposées dans le document de référence, les indicateurs CE et CIP ont été combinés afin de construire une mesure globale de vraisemblance de la classification. Une interprétation simple a été retenue en associant à chaque indicateur une quantité comprise entre 0 et 1 : 1-CE et 1-CIP. La vraisemblance globale est alors

définie comme le produit de ces deux termes. Cette mesure favorise les classifications présentant peu de contours et peu de pixels isolés.

Optimisation des poids spectraux

```
def optimize_weights(data_norm, gt, n_clusters=16, n_iter=200, seed=0):  
    """  
    Recherche aléatoire des poids c_k (un par bande spectrale).  
    À chaque itération : - on tire aléatoirement de nouveaux poids - on calcule la vraisemblance (CE + CIP) | on garde les poids qui maximisent la vraisemblance  
    """  
    rng = np.random.RandomState(seed)  
    K = data_norm.shape[2]  
  
    # Initialisation : tous les poids à 1  
    best_weights = np.ones(K)  
    X0 = build_features(data_norm, best_weights)  
    best_classif = run_kmeans(X0, n_clusters)  
    best_score = compute_vraisemblance(best_classif)  
  
    history_ce = [compute_CE(best_classif)]  
    history_oac = [clustering_accuracy(gt, best_classif)]  
    history_score = [best_score]  
  
    print(f"Iter 000 | CE={history_ce[0]:.4f} | "  
          | f"OAC={history_oac[0]:.4f} | score={best_score:.4f}")  
  
    for t in range(1, n_iter + 1):  
        # Tirage de nouveaux poids : valeurs positives entre 0 et 2  
        new_weights = rng.uniform(0, 2, size=K)  
  
        # Construction des features et clustering  
        X_new = build_features(data_norm, new_weights)  
        classif = run_kmeans(X_new, n_clusters, seed=t)  
        score = compute_vraisemblance(classif)  
  
        # Mise à jour si amélioration  
        if score > best_score:  
            best_score = score  
            best_weights = new_weights  
            best_classif = classif  
            print(f"Iter {t:03d} | CE={compute_CE(classif):.4f} | "  
                  | f"OAC={clustering_accuracy(gt, classif):.4f} | "  
                  | f"score={score:.4f} ✓ amélioration")  
  
            history_ce.append(compute_CE(classif))  
            history_oac.append(clustering_accuracy(gt, classif))  
            history_score.append(score)  
  
    return best_classif, best_weights, history_ce, history_oac, history_score
```

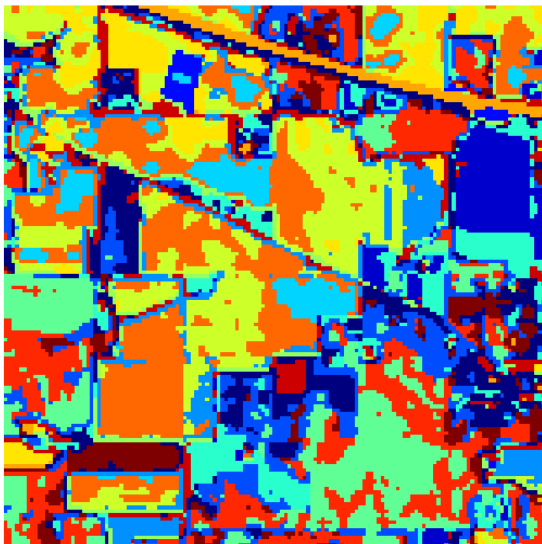
Afin d'explorer l'idée proposée dans le document de référence, une procédure d'optimisation des poids spectraux c_k a été implémentée. Chaque bande spectrale se voit attribuer un poids positif modifiant son influence dans la construction des vecteurs de caractéristiques. À chaque itération, un nouveau vecteur de poids est généré aléatoirement, puis utilisé pour effectuer une classification K-means. La qualité de cette classification est ensuite évaluée à l'aide de la mesure de vraisemblance basée sur les

indicateurs CE et CIP. Les poids conduisant à la plus grande vraisemblance sont conservés.

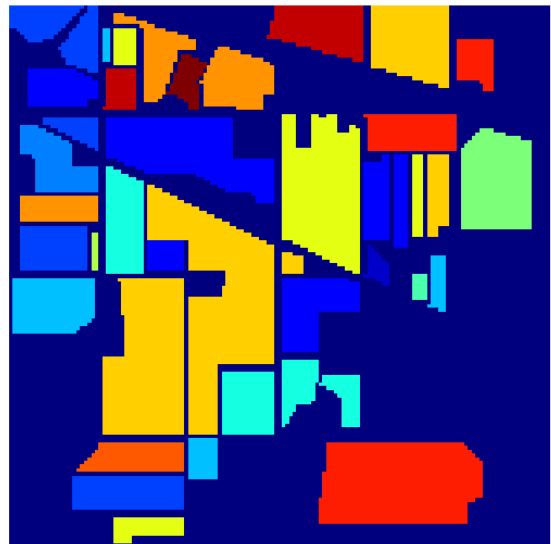
Simulation

Une première expérimentation a été réalisée en optimisant les poids spectraux ck à l'aide de la mesure de vraisemblance construite à partir des indicateurs CE et CIP. Après 200 itérations de recherche aléatoire, la mesure CE est passée de 0,6790 à 0,6349 tandis que l'OAC est passé de 0,3955 à 0,4170. Cette évolution suggère qu'une amélioration de la cohérence spatiale de la classification peut s'accompagner d'une amélioration de la qualité réelle mesurée par le ground truth. Ces premiers résultats restent toutefois préliminaires et nécessitent des expérimentations complémentaires avec des mesures de vraisemblance plus élaborées.

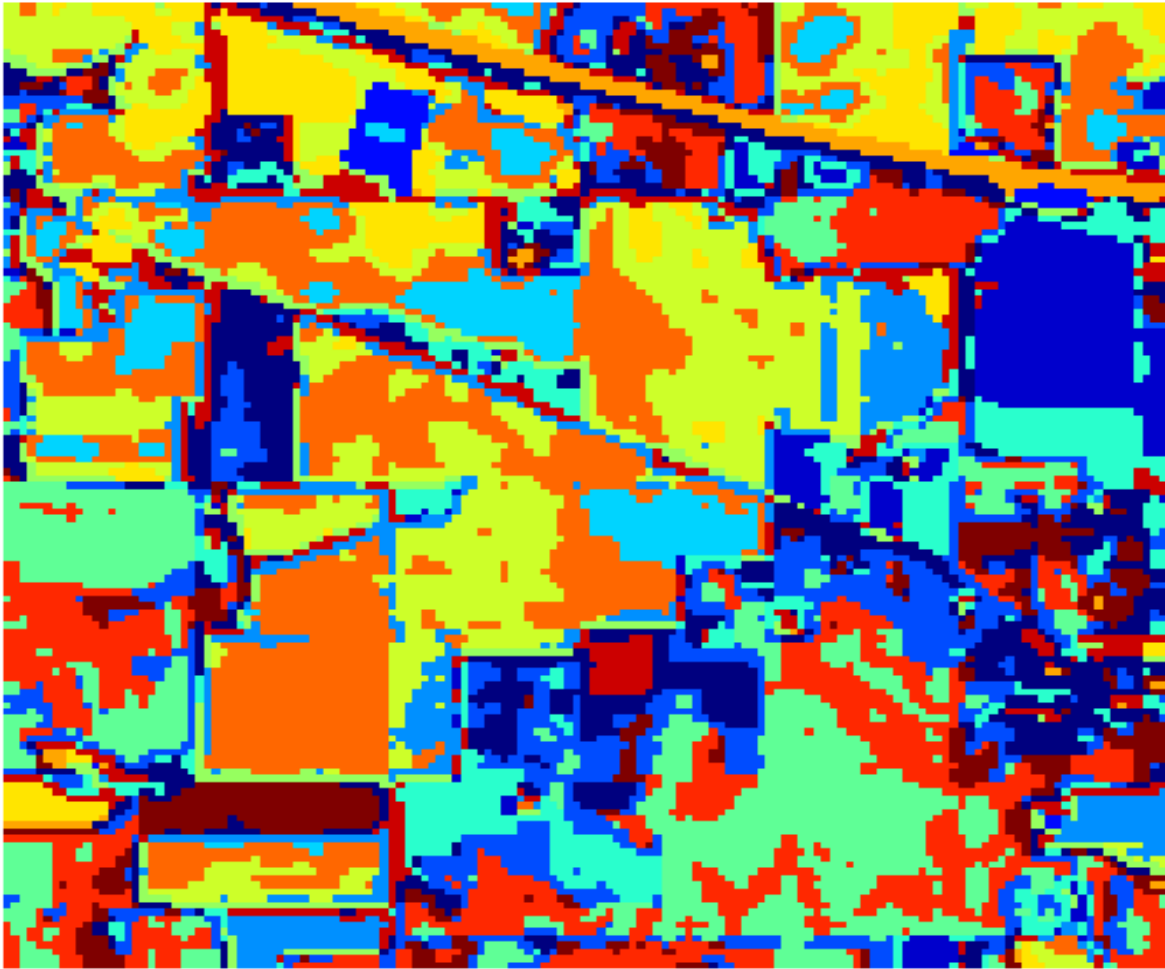
Meilleure classification
OAC=0.4170 | CE=0.6349



Ground Truth



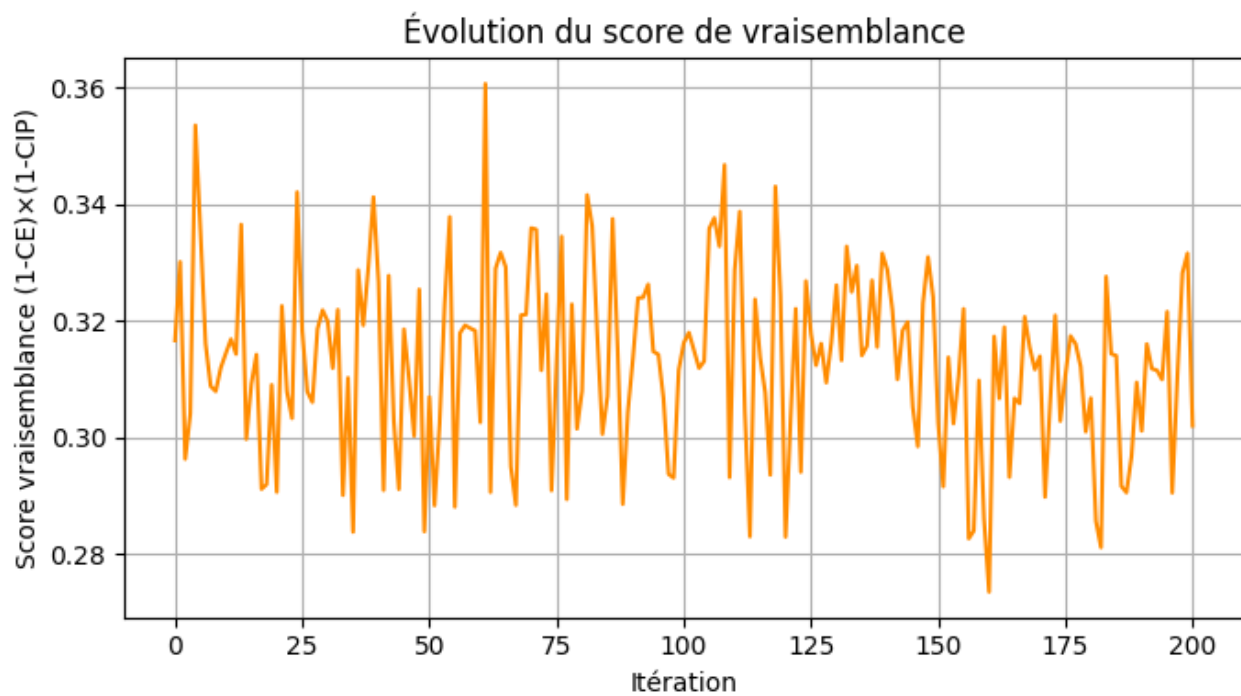
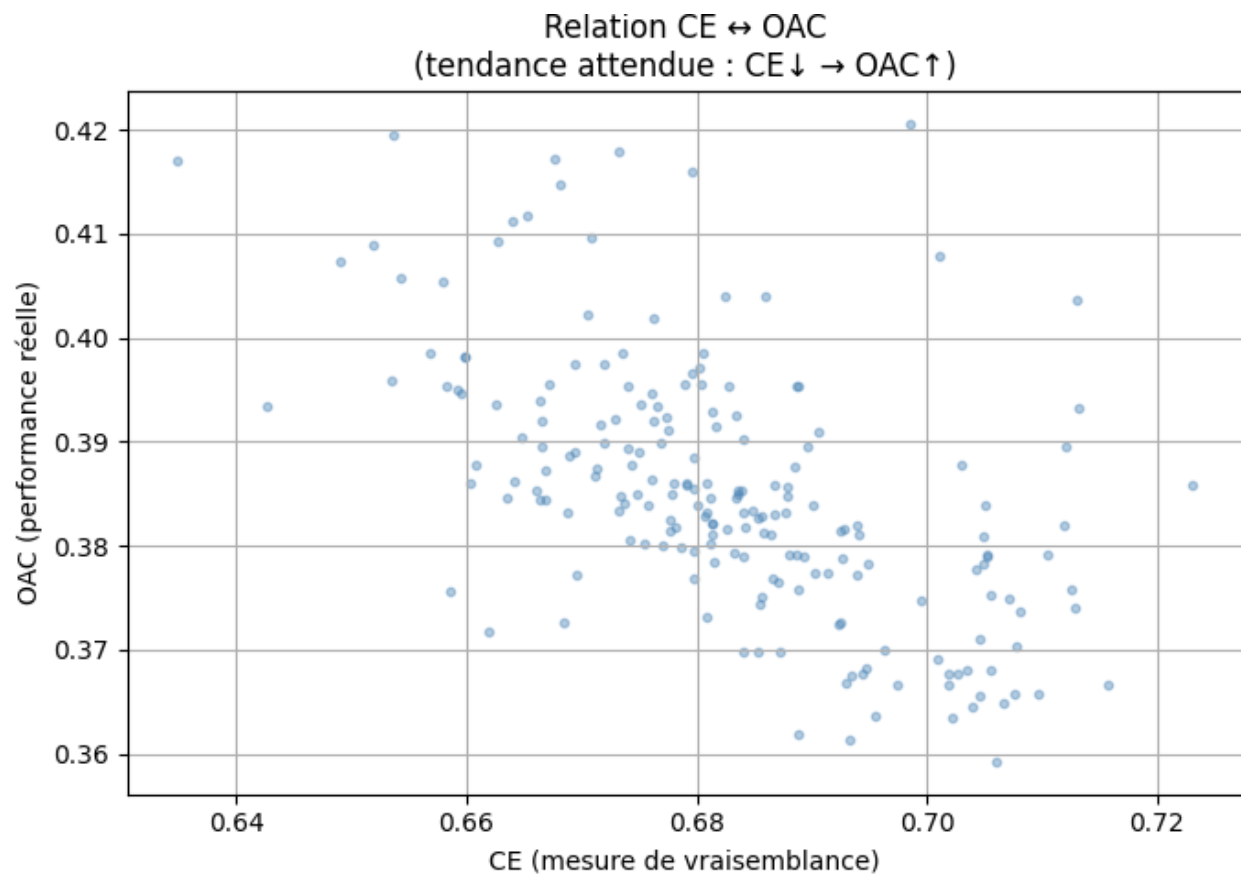
Meilleure classification
OAC=0.4170 | CE=0.6349



La figure présente la meilleure classification obtenue après l'optimisation des poids spectraux ck à l'aide de la mesure de vraisemblance basée sur les indicateurs CE et CIP. On observe que plusieurs structures spatiales présentes dans le ground truth sont partiellement retrouvées, notamment certaines grandes régions homogènes. Toutefois, la classification reste fortement fragmentée et contient encore de nombreux contours internes.

```
PS C:\Users\THIAO Assane\Documents\Internship\code> python .\kmeans_vers2.py
=== Optimisation en cours ===
Iter 000 | CE=0.6790 | OAC=0.3955 | score=0.3167
Iter 001 | CE=0.6652 | OAC=0.4116 | score=0.3301 ✓ amélioration
Iter 004 | CE=0.6426 | OAC=0.3934 | score=0.3535 ✓ amélioration
Iter 061 | CE=0.6349 | OAC=0.4170 | score=0.3607 ✓ amélioration

=== Résultat final ===
OAC = 0.4170 | CE = 0.6349
```



Je reste disponible pour toute remarque ou suggestion concernant ce travail. Je suis également disponible pour un échange ce jeudi à 10h si cela vous convient.

